🪟 Window & Menu State Management in MLX Python

Why This Topic Is Important

One of the first things students try to build in MLX projects is:

- a main menu
- a settings menu
- a gameplay screen
- a game over screen
- a theme selection screen

At first, this seems simple.

However, many beginners quickly run into problems such as:

- multiple windows opening accidentally
- images overlapping incorrectly
- menus not disappearing
- hooks stopping after changing screen
- crashes when reloading assets
- duplicated rendering loops
- freezing windows

Understanding how MLX window management works is essential for creating structured graphical projects.

⚠️ Important Concept

In most MLX projects:

```
YOU SHOULD ONLY HAVE ONE WINDOW
```

Instead of creating new windows for every menu:

❌ Wrong idea:

```
Main Menu Window
Gameplay Window
Settings Window
Theme Window
```

The recommended approach is:

✅ One MLX window ✅ Multiple GAME STATES

This is how most games work internally.

🧠 What Is a Game State?

A game state represents:

```
What screen the game is currently showing
```

Examples:

```
MENU
PLAYING
SETTINGS
THEME_SELECTION
GAME_OVER
EXIT
```

Instead of opening new windows:

the program changes its CURRENT STATE.

📦 Basic State Architecture

A very common MLX architecture uses:

```
class GameState:
    current_state = "MENU"
```

Then:

```
if state.current_state == "MENU":
    draw_menu()

elif state.current_state == "PLAYING":
    draw_game()
```

This allows the same window to display completely different screens.

🎮 Example Flow

Example:

```
Program Starts
      ↓
MENU
      ↓
Press "1"
      ↓
PLAYING
      ↓
Press "ESC"
      ↓
MENU
      ↓
Press "2"
      ↓
THEME MENU
```

The window never changes.

Only the rendered content changes.

🪟 Why Multiple Windows Are Usually a Bad Idea

MLX can technically create multiple windows.

However, for beginner projects this often creates:

- duplicated hooks
- rendering confusion
- event problems
- focus issues
- memory issues
- synchronization problems

For most 42 projects:

✅ one window is cleaner ✅ easier to debug ✅ easier to maintain

---

🧱 Recommended Project Structure

A clean menu system usually separates:

```
render/
│
├── menu.py
├── game.py
├── settings.py
├── themes.py
└── ui.py
```

🎨 Rendering Different Screens

Example:

```
if state.current_state == "MENU":
    draw_menu(mlx)

elif state.current_state == "PLAYING":
    draw_game(mlx)

elif state.current_state == "SETTINGS":
    draw_settings(mlx)
```

Each screen has its own renderer.

This keeps the project organized.

⌨️ Changing Between Menus

The keyboard hook usually changes states.

Example:

```
if key == KEY_1:
    state.current_state = "PLAYING"

elif key == KEY_2:
    state.current_state = "SETTINGS"
```

Nothing else changes.

The renderer automatically starts drawing the new screen.

🔄 The Main Rendering Loop

Most MLX projects repeatedly redraw the current state.

Conceptually:

```
while running:
    draw_current_state()
```

Example logic:

```
if state.current_state == "MENU":
    draw_menu()

elif state.current_state == "PLAYING":
    draw_game()
```

🧼 Clearing Old Screens

One of the most common beginner problems:

```
OLD IMAGES STAY ON SCREEN
```

This happens because MLX does not automatically clear old renders.

You usually need to:

- redraw the background
- redraw the entire scene
- overwrite previous images

Example order:

```
1. Draw background
2. Draw maze
3. Draw player
4. Draw UI
```

🖼️ Menu Background Systems

Menus usually use:

- static backgrounds
- banners
- title images
- option text
- overlays

Example:

```
mlx.put_image(background, 0, 0)
mlx.put_image(banner, 200, 50)
```

Then draw menu options.

🎯 Highlighting Selected Options

Many menu systems visually highlight the selected option.

Example:

```
> PLAY
  SETTINGS
  EXIT
```

This can be done using:

- different colors
- arrows
- animated selectors
- highlighted images

🧩 Recommended Menu Architecture

A good menu system normally contains:

State Variables

Example:

```
selected_option = 0
```

Navigation Hooks

Example:

```
KEY_UP
KEY_DOWN
KEY_ENTER
```

Renderer

Responsible for:

- drawing menu options
- drawing background
- highlighting selection

🎮 Example Menu Navigation Logic

Example:

```
if key == KEY_UP:
    selected_option -= 1

elif key == KEY_DOWN:
    selected_option += 1
```

Then:

```
if selected_option < 0:
    selected_option = max_options - 1
```

This creates cyclic navigation.

⚠️ Common Beginner Mistake

A very common mistake:

```
mlx_new_window()
```

every time a menu changes.

This usually creates:

- duplicated windows
- memory leaks
- invisible windows
- crashes
- event hook conflicts

The correct approach is:

✅ create ONE window ✅ change STATES

🎨 Theme Switching Systems

Many MLX projects use themes.

Example:

```
LIGHT MODE
DARK MODE
GOTHIC MODE
```

Normally:

- the state changes
- assets reload
- different textures are rendered

Example:

```
if current_theme == "DARK":
    wall_texture = dark_wall
```

🔄 Transition Effects Between Menus

Advanced projects may add transitions.

Examples:

- fade effects
- sliding menus
- animated overlays
- vortex transitions
- zoom effects

Important:

Most MLX transitions are fake animations.

They are usually created by:

- redrawing frames quickly
- changing image positions
- changing alpha-like effects manually

🐍 State Classes vs Simple Variables

Small projects often use:

```
current_state = "MENU"
```

Larger projects may use:

```
class GameState:
```

Advantages:

- cleaner organization
- grouped variables
- easier scaling
- easier debugging

Example:

```
class GameState:
    current_screen = "MENU"
    selected_option = 0
    current_theme = "DARK"
```

🧠 Real Example From Maze Projects

Typical MLX maze structure:

```
MENU
  ↓
GENERATE MAZE
  ↓
GAMEPLAY
  ↓
BFS VISUALIZATION
  ↓
PLAYER MODE
  ↓
END SCREEN
```

Each screen reuses:

✅ same window ✅ same MLX instance

Only:

- rendering changes
- hooks behavior changes
- assets displayed change

⚠️ Hooks Depending on State

Sometimes keys should behave differently depending on the screen.

Example:

Inside MENU:

```
Arrow Keys → Move selection
```

Inside GAMEPLAY:

```
Arrow Keys → Move player
```

This is normally solved using state checks.

Example:

```
if state.current_state == "MENU":
    menu_controls()

elif state.current_state == "PLAYING":
    gameplay_controls()
```

🔥 Common Problems

Very common issues:

- menu not disappearing
- duplicated renders
- hooks still active from previous screen
- player moving inside menus
- old textures remaining visible
- background not refreshing
- multiple loops running
- crashes after changing assets

Most of these happen because:

```
STATE MANAGEMENT IS NOT CLEAN
```

✅ Final Recommendation

For most MLX Python projects:

✅ Use ONE window ✅ Use STATE SYSTEMS ✅ Separate render logic by screen ✅ Separate menu hooks from gameplay hooks ✅ Redraw screens completely

This creates projects that are:

- cleaner
- easier to debug
- scalable
- easier to animate
- easier to maintain

It is one of the most important architectural concepts in MLX Python development.